

Chapter 6: Application & Software Security

CompTIA Security+ Study Slides

Walailak University — Assist. Prof. Dr. CJ

Learning Objectives

- Understand secure software development principles
- Describe the **Software Development Life Cycle (SDLC)**
- Explain **secure coding practices** and **common vulnerabilities**
- Identify **testing methods** for software assurance
- Recognize **application security controls** (e.g., input validation, code signing)

Why Application Security Matters

Modern attacks often target **applications**, not just infrastructure.

- 90% of security incidents exploit **software flaws**
- Breaches stem from **poor coding, weak authentication, or misconfigurations**
- Security must be built **from the start**, not added later

 *Goal:* integrate security throughout the development process.

Software Development Life Cycle (SDLC)

Definition:

A structured process for planning, creating, testing, and deploying software securely.

Phase	Security Focus
1. Planning & Requirements	Define security requirements, compliance needs
2. Design	Apply secure architecture and threat modeling
3. Implementation (Coding)	Use secure coding standards, code reviews
4. Testing	Conduct security testing, static/dynamic analysis
5. Deployment	Harden environment, manage secrets securely
6. Maintenance	Patch vulnerabilities, monitor, and update

Secure SDLC Principles

- **Security by Design** — integrate controls early
- **Least Privilege** — minimize permissions and access
- **Defense in Depth** — use multiple protective layers
- **Never Trust User Input** — validate and sanitize all data
- **Minimize Attack Surface** — remove unnecessary functionality
- **Fail Securely** — ensure safe behavior during failures
- **Authenticity & Integrity** — code signing and verification



Secure Software Development Approaches

Agile Security Integration

- Embed security tasks in each **sprint**
- Use **DevSecOps** pipelines (CI/CD + Security)
- Perform **automated testing** and **code scanning**

DevOps / DevSecOps

- Combine development + operations + security
- Continuous integration, delivery, and monitoring
- “Shift security left” — test earlier and more frequently

Threat Modeling

Threat modeling identifies and prioritizes possible vulnerabilities before coding.

Steps:

1. Identify assets and entry points
2. Map data flows and trust boundaries
3. Identify potential threats (STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege)
4. Mitigate and prioritize

 *Use tools like Microsoft Threat Modeling Tool or OWASP Threat Dragon.*



Secure Coding Practices

- Validate all input (length, type, format, range)
- Use parameterized queries (avoid SQL injection)
- Encode output to prevent **XSS**
- Manage errors securely (no stack traces or sensitive data leaks)
- Use strong cryptographic libraries (avoid custom crypto)
- Keep secrets out of code (use vaults or environment variables)

Input Validation

Goal: Ensure all user input conforms to expected patterns.

Types of Validation:

- **Client-side:** early convenience check (can be bypassed)
- **Server-side:** authoritative check, must be enforced

Example:

```
if not re.match("^[0-9]{5}$", zipcode):  
    raise ValueError("Invalid ZIP code")
```

 Validate input *before* using it in SQL, file paths, or commands.

Common Coding Vulnerabilities

Vulnerability	Description	Mitigation
Buffer Overflow	Writing data beyond allocated memory	Bounds checking, use safe libraries
Injection (SQL, LDAP, XML)	Inserting malicious commands into queries	Use parameterized queries, input validation
Cross-Site Scripting (XSS)	Injecting scripts into web pages	Encode output, sanitize input
Cross-Site Request Forgery (CSRF/XSRF)	Forcing authenticated users to perform actions	Anti-CSRF tokens, same-site cookies
Race Conditions	Exploiting timing to manipulate processes	Synchronize resource access

Testing Methods

Method	Description
Static Analysis (SAST)	Examines source code without execution (finds code flaws)
Dynamic Analysis (DAST)	Tests running application for vulnerabilities (e.g., OWASP ZAP, Burp Suite)
Fuzz Testing (Fuzzing)	Inputs random or malformed data to trigger errors or crashes
Penetration Testing	Simulates real-world attack scenarios to assess defenses
Code Review	Manual peer inspection of source for logic and security issues

Testing Categories

- **Black-Box Testing:** no knowledge of internal code (external attacker view)
- **White-Box Testing:** full code and architecture visibility (developer view)
- **Gray-Box Testing:** partial knowledge of system (insider or semi-trusted view)

 Combine all three to achieve full coverage.



Error & Exception Handling

- Prevent information leakage (no detailed system messages)
- Log exceptions securely without revealing sensitive data
- Implement generic user error pages ("An error occurred")
- Example: Catch exceptions gracefully to avoid application crash

Fail Securely: ensure the system remains in a safe state after errors.



Authentication & Session Security

- Enforce **MFA (Multi-Factor Authentication)**
- Use secure session management:
 - Regenerate session IDs after login
 - Set session timeouts
 - Mark cookies as `Secure` and `HttpOnly`
- Never store passwords in plaintext — use salted hashing (bcrypt, Argon2)

Code Signing & Integrity

- **Code Signing:** verifies authenticity and integrity of software using digital certificates
- Ensures software hasn't been altered or replaced by malware
- Commonly used for OS updates, mobile apps, and executables

Example: Windows Authenticode, macOS Notarization, Android APK Signing

Software Vulnerability Management

1. **Identify** vulnerabilities through scanning and testing
2. **Prioritize** using CVSS (Common Vulnerability Scoring System)
3. **Remediate** with patching, code fix, or configuration change
4. **Verify** fixes and document
5. **Monitor** continuously for new threats

Secure Software Supply Chain

- Verify third-party libraries and dependencies (e.g., `npm audit`, `pip-audit`)
- Use **hash verification** and **signed packages**
- Maintain **Software Bill of Materials (SBOM)**
- Apply continuous dependency scanning in CI/CD pipelines

 *A compromised dependency can compromise the entire application.*

Web Application Security Controls

- Use **Web Application Firewalls (WAF)**
- Enforce **HTTPS/TLS** for all communications
- Apply **Content Security Policy (CSP)** headers
- Implement **rate limiting** to prevent brute-force or DoS attacks
- Keep **frameworks and CMS** up to date

OWASP Top 10 (2021)

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable and Outdated Components
7. Identification and Authentication Failures
8. Software and Data Integrity Failures
9. Security Logging and Monitoring Failures
10. Server-Side Request Forgery (SSRF)

Secure Deployment & Maintenance

- Deploy only on **hardened servers**
- Use **staging environments** before production
- Automate deployments through **CI/CD pipelines**
- Perform regular vulnerability scanning
- Monitor logs and apply patches promptly

Summary

- Security must be part of **every SDLC phase**
- Follow **secure coding practices** and continuous testing
- Address **common vulnerabilities** (XSS, injection, CSRF, overflow)
- Use **automation and DevSecOps** for scalability
- Regularly train developers on **OWASP and secure frameworks**

End of Chapter 6

Next: [Chapter 7 — Network Design & Security](#)